

NQ-Shoppable

New client onboarding runbook

Nine steps to stand up an isolated NQ-Shoppable deployment for a new Shopify store: its own Cloudflare R2 bucket, its own Neon database, its own Shopify app, and its own Fly backend. Nothing is shared between clients except the codebase.

Placeholder	Meaning	Example
<client>	Lowercase slug, no spaces or hyphens inside the name	lovecovera
<Client Name>	Display name shown in the merchant's admin	NQ-Shoppable-Lovecovera
<shop>	The store's permanent myshopify handle	lovecovera

What you produce

Artifact	Name	Created in
R2 bucket	nq-shoppable-<client>	Step 1
Neon project	nq-shoppable-<client>	Step 2
Shopify app	NQ-Shoppable-<Client>	Step 3
Repo configs	fly.<client>.toml, shopify.app.<client>.toml, .env.<client>	Step 4
Fly app	nq-shoppable-<client>	Step 5

Before you start. Confirm the client has their own Cloudflare account and that you have admin access to their Shopify store. Everything else is created fresh. Two actions in this runbook cannot be undone: choosing a custom-distribution store (Step 7) and switching the R2 public URL after content exists (Step 1).

1

Cloudflare R2

In the client's own Cloudflare account

If R2 has never been used on the account you will hit an **Add R2 subscription** screen. Total due is \$0 and the free tier covers 10 GB storage, 1M writes and 10M reads per month, but Cloudflare requires a payment method on file.

1a. Create the bucket

- **R2 Object Storage > Create bucket**
- Name: `nq-shoppable-<client>` — bucket names are permanent, check the spelling
- Location **Automatic** (resolves to APAC), Storage class **Standard**

1b. Enable public access

- Open the bucket > **Settings > Public Development URL > Enable**, confirm by typing `allow`
- The resulting `https://pub-xxxx.r2.dev` is `R2_PUBLIC_URL` — **no trailing slash**
- The **Account ID** is the hex string inside the **S3 API URL** on the same Settings page

1c. Create the API token

- Back to **R2 Object Storage > Manage R2 API Tokens > Create API token**

Field	Value	Why
Token name	<code>nq-shoppable-<client></code>	Distinguishes it from other tokens later
Permissions	Object Read & Write	The app reads, writes and deletes objects; it never creates buckets
Specify buckets	Apply to specific buckets only	"All buckets" would reach into anything else in the client's account, including future buckets
TTL	Forever	An expiring token silently breaks uploads later
IP filtering	Leave empty	Fly machines have no stable outbound IPs

The result page shows three values. You need the two under **"Use the following credentials for S3 clients"**: **Access Key ID** and **Secret Access Key**. The `cfat_...` Token value above them is for Cloudflare's REST API and is not used.

The Secret Access Key is displayed once. Copy it before navigating away. If you lose it, the token has to be recreated and the Fly secrets updated.

Custom domain instead of `r2.dev`

The `r2.dev` URL is rate limited and uncached, which Cloudflare flags as unsuitable for production. A custom domain is better for a live store — but decide **before any video is uploaded**. The code stores absolute URLs in Postgres (`previewUrl = ${R2_PUBLIC_URL}/${key}`) and matches them back against the current value, so changing hosts later orphans every existing row and silently breaks deletes. With a custom domain, also add a CORS rule:

```
[{ "AllowedOrigins": ["https://<store-domain>", "https://<shop>.myshopify.com"],
  "AllowedMethods": ["GET", "HEAD"],
  "AllowedHeaders": ["*"],
  "MaxAgeSeconds": 3600 }]
```

2

Neon database

A fresh project per client

A separate database per client is not strictly required — rows are keyed by shop — but it keeps isolation and backups clean, and means one client's restore can never touch another's data.

- **New Project**, name `nq-shoppable-<client>`
- Region **AWS Asia Pacific (Singapore)**, to match the Fly `sin` region
- Postgres version: default

Variable	Which string	How to get it
DATABASE_URL	Pooled — host contains <code>-pooler</code>	Connection Details panel, pooling ON. Used by the app at runtime.
DIRECT_URL	Direct — no <code>-pooler</code>	Untick Connection pooling, or just delete the 8 characters <code>-pooler</code> from the host.

Keep `?sslmode=require&channel_binding=require` on both. Everything else in the two strings is identical — same user, password and database.

DIRECT_URL is not optional. `prisma migrate deploy` refuses to run through Neon's connection pooler. If it is missing, or if the pooled string is pasted into both, the container fails at boot before the server ever starts.

No migrations are run by hand. The container's `docker-start` runs `prisma generate && prisma migrate deploy` on every boot, which builds the schema on a fresh database automatically.

3

Shopify app

Created in the Dev Dashboard

Partner Dashboard > **Apps** > **Create app**. Migrated organisations are handed off to the Dev Dashboard. Do **not** use the store admin's **Settings** > **Apps** > **Develop apps** page — those are Admin-API-only apps with no OAuth, no app proxy and no theme extension support.

Field	Value
App name	NQ-Shoppable-<Client> (30 character limit)
App URL	<code>https://nq-shoppable-<client>.fly.dev</code>
Embed app in Shopify admin	checked
Preferences URL	blank
Webhooks API version	2026-07
Scopes	<code>read_products,write_products,write_files</code>
Optional scopes	blank
Use legacy install flow	UNCHECKED — the app uses <code>unstable_newEmbeddedAuthStrategy</code> (token exchange). Ticking this breaks the install.
Redirect URLs	<code>https://nq-shoppable-<client>.fly.dev/auth/callback</code>
POS	unchecked
App proxy — prefix	apps

Field	Value
App proxy — subpath	nq-videos
App proxy — URL	https://nq-shoppable-<client>.fly.dev

Release the version, then copy **Client ID** and **Client secret** from the app's credentials page. The App URL will 404 until Step 5 — Shopify does not verify it at creation, so this is expected.

The app proxy is the piece people forget. The storefront section fetches from /apps/nq-videos/... If the proxy is missing or misconfigured, the videos never load and every request returns 404 — with no error anywhere in the app admin.

4

Repo config files

Three files, copied from an existing client

Copy from the most recent client and change the slug. Do not run `shopify app config link` if the app was created in the Dev Dashboard — it overwrites the file and drops the `[app_proxy]` block.

File	Contents	Tracked?
<code>fly.<client>.toml</code>	Copy of <code>fly.toml</code> with <code>app = "nq-shoppable-<client>"</code>	No
<code>shopify.app.<client>.toml</code>	<code>client_id</code> , <code>application_url</code> , <code>[app_proxy]</code> , <code>[auth]</code> <code>redirect_urls</code> , <code>[webhooks]</code>	No
<code>.env.<client></code>	All 11 secret values — the ledger for Step 5	No

All three point at `https://nq-shoppable-<client>.fly.dev`. The `app-proxy` subpath `apps/nq-videos` stays the same for every client — it is scoped per app and store, so there is no collision.

The 11 values in `.env.<client>`

R2_ACCOUNT_ID	R2_ACCESS_KEY_ID	R2_SECRET_ACCESS_KEY
R2_BUCKET_NAME	R2_PUBLIC_URL	
DATABASE_URL	DIRECT_URL	
SHOPIFY_API_KEY	SHOPIFY_API_SECRET	
SHOPIFY_APP_URL	SCOPES	

These files are gitignored and dockerignored. `.env` and `.env.*` never reach GitHub or the container image. That is deliberate — but it also means there is no backup anywhere. Do not delete a client's config files; deleting them does not stop the running app, it only removes your ability to redeploy it.

Optional but recommended: set `automatically_update_urls_on_dev = false` in the client's `toml`. Left at `true`, running `shopify app dev` while that config is active silently repoints the live app at a temporary tunnel URL and breaks the client's storefront until you redeploy.

5

Fly backend

Create, set secrets, deploy

```
fly apps create nq-shoppable-<client>
```

Import the secrets straight from the `env` file. This avoids the quoting problem entirely — see the warning below.

```
Get-Content .env.<client> |
  Where-Object { $_ -match '^[A-Z0-9_]+= ' } |
  fly secrets import -a nq-shoppable-<client>

fly secrets list -a nq-shoppable-<client>
fly deploy -c fly.<client>.toml -a nq-shoppable-<client>
fly logs -a nq-shoppable-<client>
```

The Neon URLs contain &. Typed unquoted into bash or PowerShell, the value is truncated at `?sslmode=require` and Prisma fails with a confusing TLS error. Either wrap every `KEY=value` in quotes, or use `secrets import` as above and never think about it again.

Expect 11 secrets listed as **Staged** — that is normal for an app with no machines yet; they are applied on the first deploy.

Reading the boot log

A healthy first boot prints this sequence. Anything missing means stop and diagnose before continuing.

```
> prisma generate && prisma migrate deploy
Generated Prisma Client (v6.x)
Datasource "db": PostgreSQL database "neondb" at "ep-....neon.tech"  <- no -pooler
4 migrations found in prisma/migrations
Applying migration `..._add_video_products`  <- no-op on a fresh DB
Applying migration `..._init`
Applying migration `..._add_video_preview_url`
Applying migration `..._ensure_video_products`
All migrations have been successfully applied.
[react-router-serve] http://localhost:3000
```

Two harmless warnings on every first deploy. "The app is not listening on the expected address" during deploy, and one "instance refused connection" from the proxy. Both are timing artifacts of migrations running before the server binds to port 3000. If the log sequence above completes, ignore them.

Fly creates **two** machines by default for high availability; `min_machines_running = 1` is a floor, not a cap. Run `fly scale count 1 -a nq-shoppable-<client>` to match a single-machine setup. The second machine correctly reports *"No pending migrations to apply"* — there is no double-apply race.

Always pass `-c fly.<client>.toml`. Without it Fly falls back to the default `fly.toml`, which belongs to a different client.

6 Push config and theme extension

shopify app deploy

```
shopify app config use <client>      # the FILE suffix, not the app name
shopify app deploy
```

The CLI automatically picks up `.env.<client>` because the filename matches the config name, and uses it to resolve the Org and App. Confirm the app it names is the right one before releasing.

Reading the release diff

Line	Meaning	Action
access_scopes (updated)	Normal	Proceed
webhooks (updated)	Normal — the Dev Dashboard form has no webhook section, so these arrive from the toml	Proceed
+ shoppable-videos (new)	The theme app extension registering for the first time	Proceed
app_proxy removed	The toml is missing its [app_proxy] block	Cancel and fix

Theme-check may warn about RemoteAsset for Google Fonts and the jsDelivr Swiper CDN. These are expected and non-blocking.

7

Distribution and install

The one irreversible step

Dev Dashboard › the app › **Distribution** › **Custom distribution** › **Select**. Public distribution would require App Store review and is not what you want. Then Overview › **Installs** card › the **copy icon** next to "Install app" to get the custom install link.

Custom distribution is permanent and single-store. The app binds to whichever store the browser session belongs to when the link is opened, and Shopify will not let you change it afterwards. Installing to a dev store by accident means creating a replacement app from scratch.

Safe install order

- Open an **incognito window**
- Sign in to `https://admin.shopify.com/store/<shop>/` — the client's store, not your dev store
- Confirm you are in the right admin
- Paste the install link **in that same window**
- Review the three scopes, then **Install app**

If the client's store is not listed in your Partner organisation, that is fine and does not block anything — custom install links work on any store you can sign in to.

8

Add the section to the theme

In the client's store admin

- **Online Store** › **Themes** › **Customize**
- Select the template where the videos should appear (home page or product page)
- **Add section** › scroll to the **Apps** group › **Shoppable Videos**
- Position it, then **Save**

The section renders empty until Step 9. That is expected, not a failure.

9

Content and handover

Upload, verify, hand over

Upload videos and attach products in the app admin. Preview clips are generated automatically into this client's R2 bucket on upload — there is no backfill step on a fresh store.

Final verification checklist

Check	How	Expected
App installed	Dev Dashboard Installs card	0 changes to 1
OAuth works	The embedded app opens in the client's admin	App UI loads, no error screen
Database reachable	A row appears in the Session table in Neon	One session row for the shop
Backend healthy	fly status -a nq-shoppable-<client>	Machines started, checks passing
Upload path works	Upload one test video in the app admin	Object appears in the R2 bucket
Proxy works	Load the storefront page with the section	Videos render, no 404 on /apps/nq-videos
Preview clips	Check previews/ prefix in R2 after upload	A small .mp4 per video

Appendix — known gotchas

Cloudflare R2

- The Secret Access Key is shown once. The `cfat_...` token value is a different credential and is not used by the app.
- `R2_PUBLIC_URL` must have no trailing slash — keys are appended directly.
- Scope the token to one bucket. "All buckets in this account, including newly created" is the default and is broader than needed.
- Switching the public URL after content exists orphans every stored row. Decide at Step 1.

Neon and Prisma

- Pooled and direct strings differ by exactly the 8 characters `-pooler` in the host.
- `DIRECT_URL` is required — migrations cannot run through the pooler.
- Migrations are applied in lexicographic folder order, not timestamp intent. The `add_video_products` migration sorts before `init` and `no-ops` on a fresh database; `ensure_video_products` re-adds the column afterwards. Do not renumber these.
- Neon computes auto-suspend when idle, so the first request after a quiet period is slow. This is not a bug.

Fly

- Always pass `-c fly.<client>.toml`, or Fly uses the default `fly.toml` belonging to another client.
- Secrets show as **Staged** until the first deploy creates a machine.
- On Windows, `flyctl`'s auto-updater can fail with a missing `wintun.dll` and then refuse to run because it considers itself out of date. Fix: `iwr https://fly.io/install.ps1 -useb | iex`, then open a new shell.
- Two machines are created by default. Use `fly scale count 1` to match a single-machine setup.

Shopify

- `shopify app config use <client>` takes the config **file suffix**, not the app's display name.
- Do not run `shopify app config link` against a Dev-Dashboard-created app unless you intend to; it overwrites the `toml` and drops `[app_proxy]`.
- Leave **Use legacy install flow** unchecked. The app runs `unstable_newEmbeddedAuthStrategy` (token exchange).
- The store admin's **Develop apps** page creates Admin-API-only apps. Those cannot do app proxy or theme extensions, and new legacy custom apps cannot be created after 1 January 2026.

Repo hygiene

- `.env` and `.env.*` are gitignored and dockerignored. Per-client `tomls` are untracked too — nothing is on GitHub, so there is no undo for a deletion.
- Deleting a client's config files does not stop their running app. It only removes your ability to redeploy or update it.
- If `git status` shows every file as modified, that is usually CRLF line-ending churn from a Windows or OneDrive path — worth fixing before relying on git as a safety net.

Each client gets its own R2 bucket, Neon project, Shopify app and Fly app. The only shared thing is this codebase and the app-proxy subpath, which is scoped per app and store. Nothing you do for one client can affect another.